

Curso de Especialización Ciberseguridad en las TIC

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

Objetivos

- Mapear dónde se almacenan datos de salud
- Ver qué controles se han aplicado y qué riesgos existen
- Confeccionar la plantilla de evaluación de riesgos
- Mejorar el código para que la probabilidad sea menor

Resumen

Autora: Marina del Pilar López Oliva

Enero - 2026

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Índice de Contenidos

1. Mapear.....	4
2. Controles aplicados y riesgos existentes.....	13
3. Plantilla evaluación de riesgos.....	14
4. Código mejorado.....	15

1. Mapear

Empieza indicando la base de datos dónde se guardarán todos los datos de usuarios y pacientes. Es decir, guarda los datos sensibles.

Nombre del fichero de base de datos donde se guardan usuarios y pacientes.

NOMBRE_BD = "pacientes.db"

Tenemos una función que es "generar_hash_contrasena". Esta función genera un hash seguro de la contraseña. Se almacena un dato sensible importante, el hash de la contraseña.

def generar_hash_contrasena(contrasena: str) -> bytes:

"""Genera un hash seguro de la contraseña usando bcrypt."""

contrasena_bytes = contrasena.encode("utf-8")

sal = bcrypt.gensalt()

return bcrypt.hashpw(contrasena_bytes, sal)

En la siguiente función se comprueba si la contraseña introducida coincide con el hash que hay almacenado. Esto contiene dos datos, el hash de la contraseña real y el hash de la contraseña introducida.

def comprobar_contrasena(contrasena: str, hash_guardado: bytes) -> bool:

"""Comprueba si la contraseña introducida coincide con el hash almacenado."""

contrasena_bytes = contrasena.encode("utf-8")

return bcrypt.checkpw(contrasena_bytes, hash_guardado)

Después se tiene la función que cifra un texto sensible introducido, como por ejemplo un diagnóstico médico o datos sensibles de salud.

def cifrar_texto_sensible(texto: str) -> bytes:

"""Cifra un texto sensible (p. ej. diagnóstico médico)."""

return cifrador.encrypt(texto.encode("utf-8"))

Con la siguiente función se descifra el texto sensible para que se pueda leer sin problemas.

```
def descifrar_texto_sensible(token: bytes) -> str:
```

```
    """Descifra un texto sensible y devuelve la cadena original."""
```

```
    return cifrador.decrypt(token).decode("utf-8")
```

Seguimos con la función utilizada para inicializar la base de datos. El dato importante es el nombre de la base de datos.

```
def inicializar_base_datos():
```

```
    """Crea tablas para usuarios (profesionales) y pacientes, si no existen."""
```

```
    conexion = sqlite3.connect(NOMBRE_BD)
```

```
    cursor = conexion.cursor()
```

Después se crean dos tablas. Una tabla con la información de los empleados, son los usuarios, e incluyen un id única, un nombre, un email, el hash de la contraseña y el rol (admin o médico). La otra tabla contendrá información de los pacientes, como los datos identificativos y los datos de salud cifrados. Esta tabla contendrá un único id por paciente, el nombre, el documento identificativo como ID o NIF, el teléfono del paciente, el diagnóstico cifrado, y dónde y por quién fue creado.

```
# Tabla de usuarios profesionales (gestores de pacientes)
```

```
cursor.execute(
```

```
    """
```

```
CREATE TABLE IF NOT EXISTS usuarios (
```

```
id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
nombre TEXT NOT NULL,
```

```
email TEXT NOT NULL UNIQUE,
```

```
hash_contrasena BLOB NOT NULL,
```

```
rol TEXT NOT NULL -- 'admin' o 'medico'
```

```
);
```

```
    """
```

```
)  
  
# Tabla de pacientes: datos identificativos + datos de salud cifrados  
  
cursor.execute(  
  
    """  
  
    CREATE TABLE IF NOT EXISTS pacientes (  
  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
  
    nombre TEXT NOT NULL,  
  
    documento TEXT NOT NULL, -- DNI/NIF, etc.  
  
    telefono TEXT NOT NULL,  
  
    diagnostico_cifrado BLOB NOT NULL,  
  
    creado_en TEXT NOT NULL,  
  
    creado_por INTEGER NOT NULL,  
  
    FOREIGN KEY (creado_por) REFERENCES usuarios (id)  
  
    );  
  
    """)  
  
)  
  
conexion.commit()  
  
cursor.close()  
  
conexion.close()
```

A continuación tenemos una función para registrar usuarios de los empleados. Se introducirá el nombre, el email, la contraseña y el rol.

```
def registrar_usuario_profesional():  
  
    """Proceso: registro de usuario profesional (admin o médico) con contraseña  
    protegida."""  
  
    print("=== Registro de usuario profesional ===")
```

```
nombre = input("Nombre: ")
```

```
email = input("Email: ")
```

```
contrasena = input("Contraseña: ")
```

```
rol = input("Rol (admin/medico): ").strip().lower()
```

Después se compara si el rol introducido es admin o médico, si no es ninguna de esos dos, saldrá un aviso con que el rol no es válido.

```
if rol not in ("admin", "medico"):
```

```
    print("Rol no válido. Debe ser 'admin' o 'medico'.\n")
```

```
    return
```

A continuación, se creará el hash de la contraseña introducida para almacenarla.

```
hash_contrasena = generar_hash_contrasena(contrasena)
```

Por último, conectamos con la base de datos e intentará introducir el usuario introducido si no hay ningún error. Aquí están todos los datos sensibles del empleado.

```
conexion = sqlite3.connect(NOMBRE_BD)
```

```
cursor = conexion.cursor()
```

```
try:
```

```
    cursor.execute(
```

```
        "INSERT INTO usuarios (nombre, email, hash_contrasena, rol) "
```

```
        "VALUES (?, ?, ?, ?);",
```

```
        (nombre, email, hash_contrasena, rol),
```

```
    )
```

```
    conexion.commit()
```

```
    print(f"Usuario {nombre} registrado con rol {rol}.\n")
```

```
except sqlite3.IntegrityError:
```

```
    print("Ya existe un usuario con ese email.\n")
```

finally:

cursor.close()

conexion.close()

La función siguiente sirve para iniciar sesión con el usuario deseado. En esta función introducimos datos sensibles del empleado como el email y la contraseña en texto plano. Después se cifrará y se comparará el hash con el que está almacenado.

def iniciar_sesion_usuario():

"""Proceso: autenticación segura de un usuario profesional."""

print("=== Iniciar sesión ===")

email = input("Email: ")

contrasena = input("Contraseña: ")

conexion = sqlite3.connect(NOMBRE_BD)

cursor = conexion.cursor()

cursor.execute(

"SELECT id, nombre, hash_contrasena, rol FROM usuarios WHERE email = ?;"

(email,)

)

fila = cursor.fetchone()

cursor.close()

conexion.close()

if fila is None:

print("Usuario no encontrado.\n")

return None

id_usuario, nombre, hash_contrasena, rol = fila

if comprobar_contrasena(contrasena, hash_contrasena):


```
print(f"Bienvenido, {nombre} ({rol}).\n")

return {"id": id_usuario, "nombre": nombre, "rol": rol}

else:

print("Contraseña incorrecta.\n")

return None
```

Seguimos con la gestión de pacientes. Tenemos una función para dar de alta a los pacientes. Primero el empleado debe iniciar sesión, después se introducen los datos personales del paciente necesarios, y el diagnóstico médico que es un dato sensible.

```
def dar_alta_paciente(usuario_actual):

    """

    Proceso: alta de paciente.

    - Requiere usuario autenticado.

    - El diagnóstico médico (dato sensible) se cifra antes de guardarlo.

    """

    if usuario_actual is None:

        print("Debes iniciar sesión primero.\n")

        return

    print("=== Alta de paciente ===")

    nombre_paciente = input("Nombre del paciente: ")

    documento_paciente = input("Documento (DNI/NIF): ")

    telefono_paciente = input("Teléfono: ")

    diagnostico = input("Diagnóstico médico (dato sensible): ")

    A continuación, se cifra el contenido del diagnóstico médico para que sea
    confidencial.

    diagnostico_cifrado = cifrar_texto_sensible(diagnostico)
```

Y luego se introducen todos estos datos en la base de datos en la tabla de pacientes.

```
conexion = sqlite3.connect(NOMBRE_BD)

cursor = conexion.cursor()

cursor.execute(

    """

INSERT INTO pacientes (nombre, documento, telefono,

diagnostico_cifrado, creado_en, creado_por)

VALUES (?, ?, ?, ?, ?, ?);

    """,

    (

nombre_paciente,

documento_paciente,

telefono_paciente,

diagnostico_cifrado,

fecha_creacion,

usuario_actual["id"],

),

)

conexion.commit()

cursor.close()

conexion.close()

print("Paciente creado correctamente.\n")
```

Continuamos con más datos sensibles. Al listar los pacientes, el programa primero comprueba quién tiene la sesión iniciada. Una vez iniciada la sesión de algún

empleado, se realiza el listado de pacientes. Se listan todos los pacientes registrados mostrando también los datos sensibles.

```
conexion = sqlite3.connect(NOMBRE_BD)

cursor = conexion.cursor()

cursor.execute(

"SELECT id, nombre, documento, telefono, creado_en "

"FROM pacientes ORDER BY id;"

)

filas = cursor.fetchall()

cursor.close()

conexion.close()

if not filas:

print("No hay pacientes registrados.\n")

return

print("=== Listado de pacientes ===")

for (

id_paciente,

nombre_paciente,

documento_paciente,

telefono_paciente,

fecha_creacion,

) in filas:

print(

f"{id_paciente}: {nombre_paciente} - {documento_paciente} - "

f"{telefono_paciente} (creado en {fecha_creacion})"
```

)

print()

Otra función es poder visualizar el diagnóstico del paciente que haya registrado un usuario logueado. Lo primero que hará es comprobar que haya una sesión abierta con un usuario de los empleados. Después se comprobará si el rol que tiene administrado es admin o médico, sino no podrá ver los diagnósticos. Una vez comprobado se validará la id puesta para el paciente, y si es correcta, se mostrará su diagnóstico.

nombre_paciente, diagnostico_cifrado, fecha_creacion = fila

diagnostico = descifrar_texto_sensible(diagnostico_cifrado)

print("=== Diagnóstico del paciente ===")

print(f"Paciente : {nombre_paciente}")

print(f"Creado en: {fecha_creacion}")

print("Diagnóstico:")

print(diagnostico)

print()

Por último, tenemos el menú principal, para poder acceder a las funciones designadas. Esto da pie a visualizar los datos privados o sensibles de pacientes.

while True:

print("=== Gestor de Pacientes (datos sensibles) ===")

print("1. Registrar usuario profesional")

print("2. Iniciar sesión")

print("3. Dar de alta paciente")

print("4. Listar pacientes (datos básicos)")

print("5. Ver diagnóstico de un paciente")

print("6. Salir")

2. Controles aplicados y riesgos existentes

Las contraseñas de los usuarios profesionales no se almacenan en texto plano, se almacena su hash. Se utiliza bcrypt para generar hashes seguros con sal, evitando ataques de fuerza bruta.

El diagnóstico médico, considerado dato de salud sensible, se cifra mediante cifrado simétrico. El cifrado se realiza antes de almacenar el dato en la base de datos. En la base de datos sólo se guarda el diagnóstico cifrado.

Se definen dos roles, “admin” y “medico” para los usuarios profesionales. Solo usuarios autenticados con estos roles pueden acceder a los diagnósticos cifrados. Se comprueba el rol antes de permitir el descifrado del dato sensible.

En el listado de pacientes no se muestran los datos de salud sensible, solo se presentan datos identificativos básicos. Esto reduce bastante la exposición innecesaria de información sensible.

Para cualquier operación sobre pacientes se exige que el usuario haya iniciado sesión, se controla el usuario que crea el registro mediante el campo “creado_por”.

La clave de cifrado se genera en tiempo de ejecución y se guarda en memoria. Si la aplicación se reinicia, los datos no podrán descifrarse.

El fichero de la base de datos no está cifrado. Un atacante con acceso al sistema podría copiar o manipular la base de datos. Podrían quedar los datos personales expuestos.

Solo existen dos roles y tienen los mismos permisos sobre los datos de salud.

No se registran accesos a los diagnósticos ni acciones realizadas por los usuarios. No es posible detectar accesos indebidos o abusos de privilegios.

El diagnóstico se descifra en memoria para ser mostrado. Un fallo de seguridad podría exponer el dato sensible durante un tiempo.

Podría tener ataques de inyección SQL, aunque se mantiene controlada con los parámetros

3. Plantilla evaluación de riesgos

Risk ID	Asset/Process	Data Category	Threat	Vulnerability	Impact	Likelihood	Risk Level	Existing Controls	Recommended Controls	Responsible Person	Deadline
R1	Pérdida de datos	Datos identificativos	Robo de emails, acceso a otras cuentas	Datos identificativos en texto claro	Alto - reutilizar en servicios	Bajo	Medio	Hash de contraseñas	Restringir acceso a la lista de usuarios	Responsable de Desarrollo	20/01/26
R2	Intercepción del cifrado	Hash de la contraseña y clave de cifrado	Suplantación de identidad y visualización de datos sensibles	Clave de cifrado almacenado en memoria	Alto - datos sensibles al aire	Alto	Alto	Ninguno	Guardar en un lugar seguro la clave de cifrado	Responsable de Desarrollo	20/01/26
R3	Acceso al fichero de BD	Todos los datos identificativos	Acceso físico a la base de datos con datos identificativos	Archivo sin cifrado, permisos abiertos	Alto - datos personales al aire	Alto	Alto	Ninguno	Cifrado de la base de datos, permisos de sistema de archivos restrictivos	Responsable de TI	20/01/26
R4	Abuso de privilegios	Todos los datos cifrados y sin cifrar	Filtración de datos sensibles	Roles con permisos no controlados	Medio - visualizar datos	Alto	Alto	Roles admin y medico	Controlar los permisos de los roles	Responsable de TI	20/01/26
R5	Accesos no controlados	Todos los datos	Visualización no controlada de los datos	Accesos sin controlar	Alto - pérdida de datos	Medio	Medio	Ninguno	Meter un control de accesos y un control de visualizaciones	Responsable de TI	20/01/26
R6	Conexión y consultas a la BD	Todos los datos	Inyección SQL	Consultas con concatenación de strings	Alto - pérdida de datos	Bajo	Medio	Uso de parámetros ?	Mantener los parámetros, validación	Responsable de Desarrollo	20/01/26

4. Código mejorado

```
# ===== CONFIGURACIÓN MEJORADA =====
```

```
import os
```

```
import sqlite3
```

```
from datetime import datetime
```

```
import bcrypt
```

```
from cryptography.fernet import Fernet
```

```
from cryptography.hazmat.primitives import hashes
```

```
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
```

```
import base64
```

```
import logging
```

```
from typing import Optional
```

```
# Configuración de logging
```

```
logging.basicConfig(
```

```
    level=logging.INFO,
```

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

```
format='%(%asctime)s - %(levelname)s - %(message)s';

handlers=[

    logging.FileHandler('audit.log'),

    logging.StreamHandler()

]

)
```



```
# Gestión segura de claves (versión mejorada)

def obtener_clave_cifrado() -> bytes:

    """

    Obtiene la clave de cifrado de forma segura.

    En producción, usaría KMS, Vault o variables de entorno cifradas.

    """

    # Opción 1: Variable de entorno (mejor que en código)

    clave_env = os.getenv("CLAVE_CIFRADO_PACIENTES")

    if clave_env:
```


EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

```
return base64.urlsafe_b64decode(clave_env)
```

```
# Opción 2: Derivar de passphrase (para desarrollo)
```

```
# EN PRODUCCIÓN: Usar servicio de gestión de claves
```

```
passphrase = os.getenv("CIFRADO_PASSPHRASE", "default-passphrase-change-me")
```

```
salt = b'fixed-salt-change-in-production' # En prod, usar salt aleatorio almacenado
```

```
kdf = PBKDF2HMAC(
```

```
    algorithm=hashes.SHA256(),
```

```
    length=32,
```

```
    salt=salt,
```

```
    iterations=100000,
```

```
)
```

```
return base64.urlsafe_b64encode(kdf.derive(passphrase.encode()))
```

```
# Configuración
```

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

```
NOMBRE_BD = "pacientes.db"
```

```
CLAVE_CIFRADO = obtener_clave_cifrado()
```

```
cifrador = Fernet(CLAVE_CIFRADO)
```

```
# Timeout de sesión (30 minutos)
```

```
SESSION_TIMEOUT_MINUTES = 30
```

```
sessions = {}
```

```
# ===== GESTIÓN DE SESIONES MEJORADA =====
```

```
class SesionUsuario:
```

```
    """Gestión segura de sesiones con timeout."""
```

```
    def __init__(self, user_id: int, nombre: str, rol: str):
```

```
        self.user_id = user_id
```

```
        self.nombre = nombre
```

```
        self.rol = rol
```

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

```
self.creada = datetime.now()
```

```
self.ultima_actividad = datetime.now()
```

```
self.token = os.urandom(32).hex()
```

```
# Log de creación de sesión
```

```
logging.info(f"Sesión creada - Usuario: {nombre}, ID: {user_id}, Rol: {rol}")
```

```
def is_valid(self) -> bool:
```

```
    """Verifica si la sesión es válida y no ha expirado."""
```

```
    tiempo_inactivo = (datetime.now() - self.ultima_actividad).total_seconds()
```

```
    return tiempo_inactivo < (SESSION_TIMEOUT_MINUTES * 60)
```

```
def update_activity(self):
```

```
    """Actualiza el tiempo de última actividad."""
```

```
    self.ultima_actividad = datetime.now()
```

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

===== RBAC MEJORADO =====

class ControlAcceso:

"""Control de acceso basado en roles con permisos granulares."""

PERMISOS = {

 'admin': {

 'ver_diagnosticos': True,

 'crear_pacientes': True,

 'listar_todos': True,

 'gestionar_usuarios': True,

 'ver_logs': True

 },

 'medico': {

 'ver_diagnosticos': True,

 'crear_pacientes': True,

 'listar_todos': True,

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

```
'gestionar_usuarios': False,  
  
'ver_logs': False  
  
,  
  
'enfermero': { # Nuevo rol  
  
    'ver_diagnosticos': False,  
  
    'crear_pacientes': False,  
  
    'listar_todos': True,  
  
    'gestionar_usuarios': False,  
  
    'ver_logs': False  
  
}  
  
}
```

@staticmethod

def tiene_permiso(rol: str, permiso: str) -> bool:

"""Verifica si un rol tiene un permiso específico."""

return ControlAcceso.PERMISOS.get(rol, {}).get(permiso, False)

```
# ===== FUNCIONES DE SEGURIDAD MEJORADAS =====
```

```
def limpiar_memoria_sensible(dato: str):
```

```
    """Intenta limpiar datos sensibles de memoria."""
```

```
    # Python no garantiza limpieza inmediata, pero es una buena práctica
```

```
    import ctypes
```

```
    strlen = len(dato)
```

```
    buffer = ctypes.create_string_buffer(strlen)
```

```
    ctypes.memset(buffer, 0, strlen)
```

```
    del dato
```

```
def cifrar_texto_sensible(texto: str) -> bytes:
```

```
    """Cifra texto sensible y limpia el original de memoria."""
```

```
    try:
```

```
        cifrado = cifrador.encrypt(texto.encode("utf-8"))
```

```
        return cifrado
```

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

finally:

Intentar limpiar el texto original de memoria

limpiar_memoria_sensible(texto)

def descifrar_texto_sensible(token: bytes) -> str:

"""Descifra texto sensible y asegura limpieza tras uso."""

texto_descifrado = cifrador.decrypt(token).decode("utf-8")

Devolver copia y programar limpieza

resultado = texto_descifrado[:]

En entorno real, usaríamos context managers para limpieza garantizada

return resultado

===== VALIDACIÓN DE ENTRADA =====

def validar_dni(dni: str) -> bool:

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

```
"""Valida formato básico de DNI español."""
```

```
import re
```

```
patron = r'^\d{8}[A-Z]$',
```

```
return bool(re.match(patron, dni))
```

```
def validar_telefono(tel: str) -> bool:
```

```
"""Valida formato de teléfono."""
```

```
import re
```

```
patron = r'^[\d\s\-\+\(\)]{9,15}$'
```

```
return bool(re.match(patron, tel))
```

```
# ===== INICIALIZACIÓN BD CON CIFRADO =====
```

```
def inicializar_base_datos_segura():
```

```
"""Inicializa BD con configuraciones de seguridad."""
```

```
conexion = sqlite3.connect(NOMBRE_BD)
```

```
cursor = conexion.cursor()
```


EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

Activar WAL mode para mejor concurrencia y consistencia

```
cursor.execute("PRAGMA journal_mode=WAL;")
```

Configurar timeout de BD

```
cursor.execute("PRAGMA busy_timeout=5000;")
```

Crear tabla de logs de auditoría

```
cursor.execute("""
```

```
CREATE TABLE IF NOT EXISTS logs_auditoria (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    timestamp TEXT NOT NULL,
```

```
    usuario_id INTEGER,
```

```
    usuario_nombre TEXT,
```

```
    accion TEXT NOT NULL,
```

```
    recurso TEXT,
```

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

```
ip_address TEXT,  
resultado TEXT,  
FOREIGN KEY (usuario_id) REFERENCES usuarios (id)  
);  
""")
```

Índices para mejor performance en búsquedas

```
cursor.execute("CREATE INDEX IF NOT EXISTS idx_pacientes_doc ON pacientes(documento);")
```

```
cursor.execute("CREATE INDEX IF NOT EXISTS idx_logs_timestamp ON logs_auditoria(timestamp);")
```

```
conexion.commit()
```

```
cursor.close()
```

```
conexion.close()
```

Establecer permisos de archivo (Unix/Linux)

```
if os.name == 'posix':
```

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

```
os.chmod(NOMBRE_BD, 0o600) # Solo owner puede leer/escribir
```

```
# ===== FUNCIÓN DE LOGGING =====
```

```
def registrar_acceso(usuario_id: Optional[int], usuario_nombre: Optional[str],
```

```
    accion: str, recurso: str, resultado: str = "OK"):
```

```
    """Registra acceso en logs de auditoría."""
```

```
    import socket
```

```
    import getpass
```

```
    timestamp = datetime.now().isoformat()
```

```
    try:
```

```
        ip = socket.gethostbyname(socket.gethostname())
```

```
    except:
```

```
        ip = "127.0.0.1"
```

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

Log a archivo y BD

logging.info(f"AUDIT - {timestamp} - User:{usuario_nombre} - Action:{accion} - Resource:{recurso} - Result:{resultado}")

conexion = sqlite3.connect(NOMBRE_BD)

cursor = conexion.cursor()

cursor.execute(

"INSERT INTO logs_auditoria (timestamp, usuario_id, usuario_nombre, accion, recurso, ip_address, resultado) VALUES (?, ?, ?, ?, ?, ?, ?)",

(timestamp, usuario_id, usuario_nombre, accion, recurso, ip, resultado)

)

conexion.commit()

cursor.close()

conexion.close()

===== VER DIAGNÓSTICO CON MEJORES CONTROLES =====

def ver_diagnostico_paciente_mejorado(usuario_actual):

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

""""Versión mejorada con controles de seguridad."""""

if usuario_actual is None:

print("Debes iniciar sesión primero.\n")

registrar_acceso(None, None, "VER_DIAGNOSTICO", "PACIENTE", "NO_AUTH")

return

Verificar permisos específicos

if not ControlAcceso.tiene_permiso(usuario_actual["rol"], "ver_diagnosticos"):

print("No tienes permisos para ver diagnósticos.\n")

registrar_acceso(usuario_actual["id"], usuario_actual["nombre"],

"VER_DIAGNOSTICO", "PACIENTE", "DENIED_PERMISSION")

return

try:

id_paciente = int(input("ID del paciente: "))

except ValueError:

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

```
print("ID no válido.\n")

registrar_acceso(usuario_actual["id"], usuario_actual["nombre"],

                 "VER_DIAGNOSTICO", f"PACIENTE_ID_INVALID", "ERROR_FORMAT")

return
```

```
conexion = sqlite3.connect(NOMBRE_BD)
```

```
cursor = conexion.cursor()
```

```
# Verificar si usuario tiene acceso a este paciente específico
```

```
# (Implementar lógica de "pacientes asignados" si es necesario)
```

```
cursor.execute(
```

```
    "SELECT nombre, diagnostico_cifrado, creado_en, creado_por FROM pacientes WHERE id = ?",
```

```
    (id_paciente,)
```

```
)
```

```
fila = cursor.fetchone()
```

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

if fila is None:

```
print("Paciente no encontrado.\n")
```

```
registrar_acceso(usuario_actual["id"], usuario_actual["nombre"],
```

```
    "VER_DIAGNOSTICO", f"PACIENTE_{id_paciente}", "NOT_FOUND")
```

```
cursor.close()
```

```
conexion.close()
```

```
return
```

nombre_paciente, diagnostico_cifrado, fecha_creacion, creado_por = fila

Usar context manager para limpieza automática

try:

```
    diagnostico = descifrar_texto_sensible(diagnostico_cifrado)
```

```
    print("=== Diagnóstico del paciente ===")
```

```
    print(f"Paciente : {nombre_paciente}")
```

EJ-4. EVALUACIÓN DE RIESGOS-GESTIÓN PACIENTES

```
print(f"Creado en: {fecha_creacion}")
```

```
print("Diagnóstico:")
```

```
print(diagnostico)
```

```
print()
```

```
# Registrar acceso exitoso
```

```
registrar_acceso(usuario_actual["id"], usuario_actual["nombre"],
```

```
    "VER_DIAGNOSTICO", f"PACIENTE_{id_paciente}", "OK")
```

```
except Exception as e:
```

```
    print(f"Error al descifrar diagnóstico: {e}")
```

```
    registrar_acceso(usuario_actual["id"], usuario_actual["nombre"],
```

```
        "VER_DIAGNOSTICO", f"PACIENTE_{id_paciente}", "ERROR_CIFRADO")
```

```
finally:
```

```
    # Limpiar recursos
```

```
    cursor.close()
```

```
    conexion.close()
```